Explainable AI in CARCI

How SHAP turns an anomaly score into a root cause

Cost Anomaly Root Cause Intelligence | Teaching guide for students

This guide explains the explainable-AI (XAI) layer of the CARCI project. CARCI detects unusual AWS cost rows with an Isolation Forest, uses SHAP to say WHICH of the seven measured features made each row look unusual, and then hands those facts to a language model that writes the root-cause explanation and the recommended action. Detection alone produces a number; these two layers are what turn that number into something a finance or engineering reader can act on.

Everything in this document is taken from the project source code or measured by running the real pipeline. Code listings are copied from the files, file and line references point at the current code, and every number in the worked example was produced by running the project on 'data set/test_2_obvious_anomalies.csv'.

What this guide covers

Part	Topic	For
1	Why an anomaly score is not an explanation	Motivation
2	Shapley values and SHAP, in one page	Theory
3	Where SHAP lives in the codebase (file map)	Code tour
4	The seven features that get explained	Code tour
5	The explanation function, line by line	Code tour
6	Reading the numbers: sign, scale, additivity	Core skill
7	A full worked example with real output	Core skill
8	From the SHAP array to the screen	Code tour
9	Root cause analysis: the LLM explanation layer	Core skill
10	When SHAP does not run: the two fallbacks	Robustness
11	Cost, limits and honest caveats	Critique
12	Lesson plan, demos and exercises	Teaching
13	Glossary and exam-style questions	Revision

Prerequisites

- Python, pandas and a first course in machine learning.
- Knowing what a decision tree is. Isolation Forest is explained here in enough depth to follow the SHAP discussion, but not from scratch.
- The project running locally: pip install -r requirements.txt, then streamlit run app.py.

Environment used for every measurement in this document: Python 3.13.5, shap 0.50.0, scikit-learn as pinned in requirements.txt, Gemini model gemini-3.5-flash-lite, Windows 11.

1. Why an anomaly score is not an explanation

The detection half of CARCI ends with one number per billing row. For the most anomalous row in our sample file that number is:

```
if_decision_score = -0.133181      # sklearn decision_function, negative = outlier
anomaly_score     =  0.133181      # the project stores the negated value
model_is_anomaly  =  1             # below the calibrated threshold
```

That is enough to raise an alert and no more. It does not say whether the row was flagged because it cost a lot, because it moved an unusual amount of data, because it scaled out far more than normal, or because of some combination. A cloud engineer cannot act on -0.133181.

The stated purpose of the project is root-cause intelligence, so the missing half is attribution: dividing the score among the input features. That is the job of the XAI layer, and in CARCI it is implemented with SHAP.

What the XAI layer produces

- A signed contribution for each of the 7 features, for every scored row (the shap_<feature> columns).
- A ranked list of the 3 strongest features per anomaly (record['top_ml_features']).
- The clause in the generated root-cause paragraph that names those features.
- The 'ML Feature Attribution' table in the Streamlit UI and the per-feature rows in the detailed CSV export.

> **The one-sentence version for students**
> Isolation Forest answers 'is this row weird?'. SHAP answers 'which measurement made it weird?'. CARCI needs both, because a cost alert without a cause is not actionable.

Where the answer appears in the product

Where the user sees it	Produced by
The root-cause paragraph, which names the top features as the drivers of the flag	carci/llm_reasoning.py (LLM), carci/analysis.py:240 (fallback)
'ML Feature Attribution' tab inside each anomaly expander	app.py:438, app.py:453
Caption 'Attribution: SHAP TreeExplainer' on the summary page	app.py:317
feature_contribution_<name> rows in the detailed CSV export	carci/reporting.py:55-65
'Attribution method:' line in the downloadable text report	carci/analysis.py:346

2. Shapley values and SHAP, in one page

SHAP (SHapley Additive exPlanations) borrows an idea from cooperative game theory. In a game, several players cooperate and produce a payout; the Shapley value is the provably fair way to split that payout among them. SHAP maps this onto machine learning:

Game theory	Machine learning	In CARCI
Players	Input features	The 7 columns in FEATURE_COLUMNS
Payout	The model's output for one row	Isolation Forest path length
Coalition	A subset of features being used	e.g. {cost, daily_z}
Shapley value	One feature's fair share of the output	shap_cost, shap_daily_z, ...

The fair share is defined as the feature's average marginal contribution over every possible ordering of the features. Informally: put the features into the model one at a time in a random order, record how much the output moves when this feature joins, and average that over all orderings.

```
phi_i  =  sum over subsets S not containing i  of

            |S|! * (M - |S| - 1)!
            --------------------  *  [ f(S + {i}) - f(S) ]
                    M!

M = number of features (7 here); f(S) = model output using only the features in S
```

Averaging over orderings is what makes the split fair rather than arbitrary. It also makes the exact computation expensive: with M features there are 2^M subsets. For CARCI's 7 features that is 128 subsets per row, which is small, but the brute-force approach also requires evaluating the model on each subset, which trees do not natively support.

Why TreeExplainer, and not the generic explainer

shap.TreeExplainer exploits the structure of tree ensembles to compute exact Shapley values in polynomial time instead of exponential time, by walking each tree once and tracking how the sample would flow down both sides of every split. This is why the project can afford to explain thousands of rows, and why the code passes the fitted forest directly:

```
explainer   = shap.TreeExplainer(model)      # model IS the fitted IsolationForest
shap_values = explainer.shap_values(x_score) # exact, per row, per feature
```

> **Important: this is not a surrogate model**
> A common misunderstanding is that SHAP trains a second, simpler model to imitate the first. TreeExplainer does not. It reads the actual trees of the actual fitted forest, so the explanation describes the model that made the decision, not an approximation of it.

The three properties worth naming in class

Property	Meaning	Why it matters here
Local accuracy (additivity)	base value + sum of all SHAP values = the model output for that row	Lets you verify the explanation arithmetically - see section 6
Missingness	A feature that is not used gets exactly 0	A constant column cannot be blamed for an anomaly
Consistency	If a model changes so a feature matters more, its SHAP value cannot go down	Rankings stay trustworthy across model retrains

SHAP explanations are local: they explain one row at a time. CARCI uses exactly this - each anomaly card carries its own attribution table. Averaging |SHAP| over many rows would give a global feature-importance view, which the project does not currently build.

3. Where SHAP lives in the codebase

The XAI layer is deliberately small and sits in one function. Everything else either feeds it or displays what it produced.

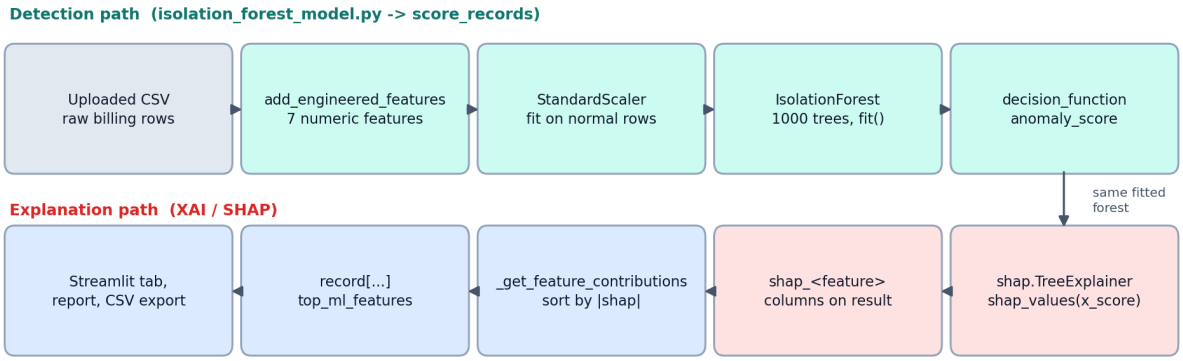


Figure 1. The green row is detection; the red and blue row is the explanation path. The explainer reads the same fitted forest that produced the score.

File	Role in the XAI layer	Key lines
carci/isolation_forest_model.py	Computes SHAP. _append_feature_attribution() is the whole XAI layer; ATTRIBUTION_METHOD is the provenance label.	33, 91-117, 120-152
carci/shared.py	Defines the 7 features that get explained and builds the scaled matrix that is both scored and explained.	10-21, 185-202
carci/analysis.py	Consumes the shap_ columns, ranks them, picks the top 3, writes them into the anomaly record and the root-cause sentence.	82-99, 176-191, 240, 256, 283-285
carci/reporting.py	Writes per-feature contributions into the detailed CSV export.	55-65
carci/llm_reasoning.py	Sends the SHAP ranking, with the rest of the anomaly facts, to Gemini as JSON and receives the written explanation. See section 9.	whole file
app.py	Displays the attribution table, the attribution-method caption and the engine that wrote the explanation.	317, 428-450
requirements.txt	Declares the dependency: shap>=0.46,<1.0	9

> **Teaching tip: read the code in this order**
> shared.py (what is measured) -> isolation_forest_model.py (how it is explained) -> analysis.py (how the explanation is ranked and worded) -> app.py (how it is shown). Following the data forwards is much easier for students than starting at the UI.

4. The seven features that get explained

SHAP can only apportion blame among the features the model actually sees. In CARCI that is the list in carci/shared.py:10-21 - seven numeric columns, four measured and three derived. Categorical context such as service, region and usage_type is NOT in the model and therefore never appears in a SHAP table.

#	Feature	Origin	Meaning
1	cost	measured	Line-item cost
2	usage_quantity	measured	Billed usage amount
3	autoscaling_count	measured	Scaling activity on the resource
4	data_transfer_gb	measured	Network transfer volume
5	cost_per_usage	derived	cost / (usage_quantity + 1), a unit-price proxy
6	transfer_per_cost	derived	data_transfer_gb / (cost + 1), transfer efficiency
7	daily_z	derived	How unusual the row's calendar day is by total spend

The derived features are built in add_engineered_features (carci/shared.py:121-128) and add_daily_context (carci/shared.py:91-118). daily_z is shared by every row on the same day, so it is the only feature that carries day-level context into an otherwise row-local model.

> **A subtlety students should notice**
> SHAP is computed on the SCALED matrix (StandardScaler, carci/shared.py:198-200), but the table shown in the UI prints the RAW feature value next to the SHAP value (carci/analysis.py:183). Row order is preserved so the pairing is correct, but the two columns are in different units: raw dollars or gigabytes next to a path-length contribution.

5. The explanation function, line by line

This is the entire XAI layer, copied from carci/isolation_forest_model.py:120-152. The numbered notes below correspond to the marked lines.

```
def _append_feature_attribution(
    result: pd.DataFrame,
    model: IsolationForest,
    x_score: object,
    train_features: pd.DataFrame,
) -> pd.DataFrame:
    try:
        import shap                                          # (1) line 127

        explainer = shap.TreeExplainer(model)                # (2) line 129
        shap_values = np.asarray(explainer.shap_values(x_score))   # (3) line 130
        # Some SHAP versions return (rows, features, outputs) for tree models;
        # collapse the trailing axis so the per-feature indexing below is valid.
        if shap_values.ndim == 3:                            # (4) lines 133-134
            shap_values = shap_values[:, :, 0]
        if shap_values.shape[1] != len(FEATURE_COLUMNS):     # (5) lines 135-139
            raise ValueError(
                f"SHAP returned {shap_values.shape[1]} columns for "
                f"{len(FEATURE_COLUMNS)} features."
            )
        result.attrs["attribution_method"] = ATTRIBUTION_METHOD    # (6) line 140
    except Exception:                                        # (7) lines 141-147
        train_center = np.asarray(train_features[FEATURE_COLUMNS].median())
        score_values = np.asarray(result[FEATURE_COLUMNS])
        train_scale = np.asarray(train_features[FEATURE_COLUMNS].std(ddof=0).replace(0, 1))
        shap_values = (score_values - train_center) / train_scale
        shap_values = shap_values * np.asarray(result["anomaly_score"]).reshape(-1, 1)
        result.attrs["attribution_method"] = "standardized feature impact fallback"

    for index, column in enumerate(FEATURE_COLUMNS):         # (8) lines 149-150
        result[f"shap_{column}"] = shap_values[:, index]

    return result
```

Note	Line	What it does and why
(1)	127	shap is imported INSIDE the function, not at module top. If the library is missing the app still runs - it drops to the fallback instead of crashing on startup.
(2)	129	The explainer wraps the fitted forest itself. No retraining, no surrogate.
(3)	130	One call explains every scored row at once. Returns an array of shape (rows, features).
(4)	133-134	Version guard. Some SHAP releases return a 3-D array for tree models; the trailing output axis is collapsed so indexing stays valid.
(5)	135-139	Shape assertion. If the column count does not match the 7 features, it raises - and the raise is caught by the except below, so a mismatch degrades instead of corrupting the table.
(6)	140	Records provenance: the string that ends up in the UI caption and the root-cause sentence.
(7)	141-147	The fallback. Any failure at all produces a standardized-distance approximation instead. See section 10.
(8)	149-150	Writes one column per feature: shap_cost, shap_usage_quantity, ... This naming is the contract the rest of the project reads.

How it is switched on

score_records() takes explain=True by default (carci/isolation_forest_model.py:95) and calls the function at lines 114-115. The CLI evaluation path never calls it, which is why 'python -m carci.isolation_forest_model dataset.csv' is fast while the Streamlit app is slow.

```
    if explain:                                          # line 114
        result = _append_feature_attribution(result, model, x_score, train_features)
```

6. Reading the numbers: sign, scale, additivity

This is the section students get wrong most often, so it is worth doing slowly. A SHAP value in CARCI is NOT a number of dollars. It is a contribution to the forest's path length for that row.

6.1 What the sign means

Isolation Forest isolates anomalies quickly: an unusual row gets separated from the rest after only a few splits, so it has a SHORT average path length. Because SHAP explains that path length, a feature that makes a row easier to isolate pushes the path length DOWN and therefore receives a NEGATIVE SHAP value.

The rule to memorise

Negative SHAP value = this feature pushed the row TOWARD anomaly.
Positive SHAP value = this feature made the row look MORE normal.
Magnitude = how strong the push was. The project ranks by magnitude alone.

This is not a claim to take on faith. Measured on 200 rows of the sample file, the sum of the seven SHAP values averages -4.0045 for rows the model flagged and 0.2026 for rows it did not, and it correlates with the raw decision_function at r = 0.997068:

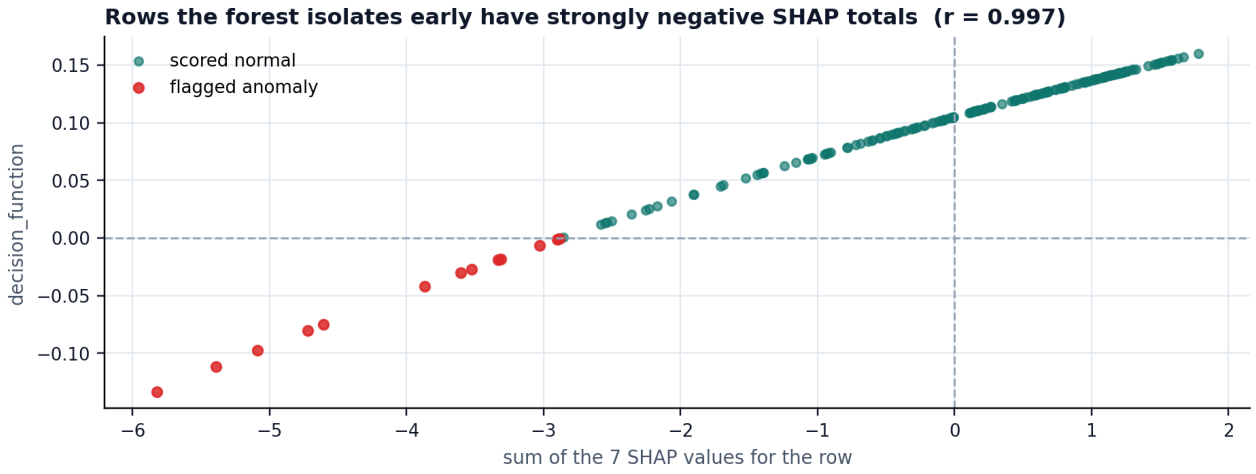


Figure 2. Every point is one billing row. Rows with strongly negative SHAP totals are exactly the rows the forest isolates early and flags as anomalies.

6.2 Additivity, verified

The local-accuracy property says base value + sum of SHAP values reproduces the model output. For Isolation Forest the output being explained is the expected path length, which scikit-learn then converts into a score. Both steps can be checked numerically:

```
base value (explainer.expected_value)      = 10.848936
sum of the 7 SHAP values for the top row   = -5.82207
base + sum                                 = 5.026866   <- expected path length E[h(x)]

sklearn then maps path length to a score:
    score_samples(x) = -2 ** ( -E[h(x)] / c(n) )
    where c(n) is the average path length of an unsuccessful BST search, c(190) = 9.6485

reconstructed vs actual score_samples: correlation 1.0, max absolute error 0.0

decision_function                          = -0.133181
score_samples                              = -0.696887
offset_ (decision_function = score - offset_) = -0.563706
```

What this proves, and what it does not

The reconstruction is exact to the last digit, so the SHAP values genuinely account for the whole model output - nothing is hand-waved. But note that base + sum equals the PATH LENGTH, not decision_function. Students who try to add SHAP values directly onto -0.133 will get nonsense; the exponential transform sits in between.

6.3 Units, and what you may legitimately say

Legitimate	Not legitimate
'cost was the strongest driver of this flag'	'cost contributed $1.96 of the overspend'
'autoscaling_count mattered nearly as much as cost'	'autoscaling_count caused 30% of the bill'
'data_transfer_gb slightly reduced the anomaly signal here'	'removing data_transfer_gb would change the cost'

7. A full worked example with real output

Everything below was produced by running the project on the first 200 rows of 'data set/test_2_obvious_anomalies.csv'. The anomaly card CARCI ranked first was AmazonRDS 2026-01-05, and this is its driver row's attribution table exactly as _get_feature_contributions builds it.

feature	feature_value (raw)	shap_value	impact_strength	direction
cost	241.33	-1.9580	1.9580	toward anomaly
autoscaling_count	12.0	-1.8355	1.8355	toward anomaly
cost_per_usage	0.2607	-1.1064	1.1064	toward anomaly
daily_z	1.6714	-0.4026	0.4026	toward anomaly
usage_quantity	924.79	-0.0939	0.0939	toward anomaly
data_transfer_gb	28.9	+0.0900	0.0900	toward normal
transfer_per_cost	0.1193	-0.0814	0.0814	toward anomaly

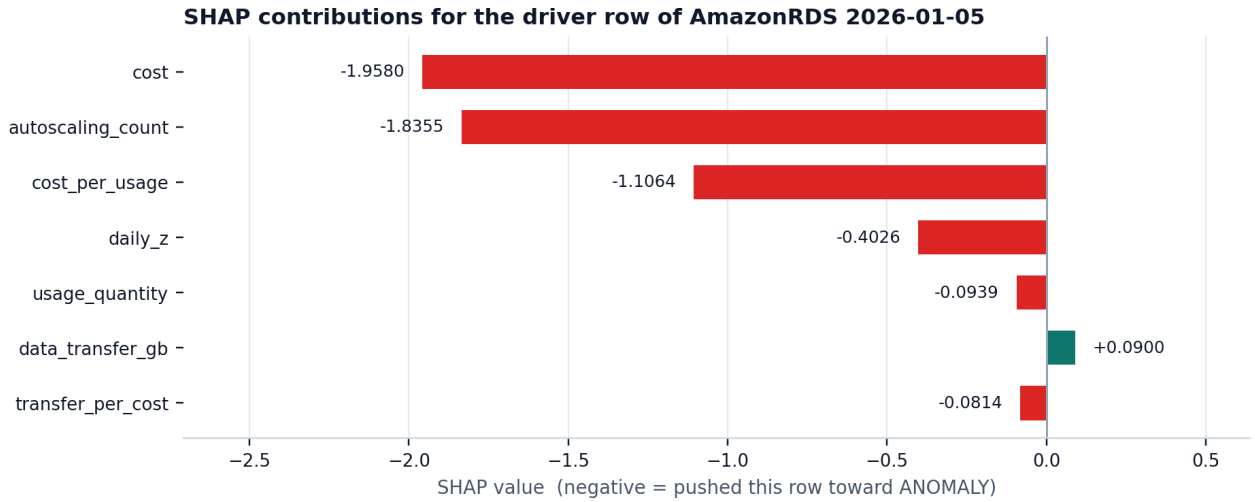


Figure 3. The same table as a chart. Bars to the left pushed the row toward being flagged; the single bar to the right pushed it back toward normal.

Reading it out loud

Cost is the largest contributor at -1.9580, with autoscaling_count almost as large at -1.8355 and cost_per_usage third at -1.1064. So this row was not flagged merely for being expensive: it was expensive AND had scaled out 12 times AND had a high unit price, and the combination is what isolated it. data_transfer_gb is the only positive value (+0.0900), meaning its transfer volume was unremarkable and mildly argued against the flag.

Note what does not appear anywhere in this table: service, region, resource or usage_type. Those are not model features. CARCI attributes them separately by cost share in the 'Factor Contributions' tab, which is arithmetic on the billing data, not machine learning. Keeping the two apart is a good exam question.

How the table becomes the sentence

The top three by impact_strength are cost, autoscaling_count, cost_per_usage. That list is stored as record['top_ml_features'] and becomes the driver clause of the written explanation. Which engine writes that sentence is the subject of section 9; either way the three names come from this table, not from a human or a guess.

```
# Fallback template wording (carci/analysis.py:240):
The Isolation Forest model flagged the driving line item as anomalous.
SHAP TreeExplainer identified the strongest feature impacts from
cost, autoscaling_count, cost_per_usage.

# LLM wording for the same record, from a real run:
The top features driving the anomaly score are autoscaling count, cost,
and cost per usage.
```

That is the whole point of the layer: the driver named in the sentence a user reads is a measured attribution, not an assumption.

8. From the SHAP array to the screen

Three short hops connect the numeric array to what the user sees. Students should be able to name all three.

Hop 1 - rank the features (carci/analysis.py:176-191)

```
def _get_feature_contributions(row: pd.Series) -> Tuple[List[Dict], List[str]]:
    feature_rows = []
    for column in FEATURE_COLUMNS:
        shap_value = float(row.get(f"shap_{column}", 0.0))
        feature_rows.append(
            {
                "feature": column,
                "feature_value": round(float(row.get(column, 0.0)), 4),
                "shap_value": round(shap_value, 4),
                "impact_strength": round(abs(shap_value), 4),
            }
        )

    feature_rows.sort(key=lambda item: item["impact_strength"], reverse=True)
    top_features = [item["feature"] for item in feature_rows[:3] if item["impact_strength"] > 0]
    return feature_rows, top_features
```

- impact_strength = abs(shap_value). Ranking is by magnitude, so direction is deliberately ignored when choosing the top three.
- row.get(..., 0.0) means a missing shap_ column silently becomes 0 rather than raising - which is what allows the degenerate path in section 9 to work.
- Only the top 3 with non-zero impact reach the sentence; the full 7 still reach the UI table.

Hop 2 - attach to the anomaly record (carci/analysis.py:256, 283-285)

```
    feature_contributions, top_features = _get_feature_contributions(row)
    ...
        "feature_contributions": feature_contributions,
        "top_ml_features": top_features,
        "attribution_method": attribution_method,
```

Hop 3 - render and export

Destination	Code	What it shows
Streamlit tab	app.py:438, 453	'ML Feature Attribution' dataframe: all 7 rows, sorted
Summary caption	app.py:317	'Attribution: SHAP TreeExplainer'
Text report	carci/analysis.py:346	'Attribution method:' line
Detailed CSV	carci/reporting.py:55-65	feature_contribution_<name> rows, top 5 features only, formatted 'value=..., shap=...'
Optional LLM prose	carci/llm_reasoning.py:110	top features are passed as facts the model may describe but not invent

> **Note the top-5 truncation**
> The UI table shows all seven features but the detailed CSV export writes only the first five (carci/reporting.py:55). If a student compares the two and finds rows missing, that slice is the reason - not a bug in the model.

9. Root cause analysis: the LLM explanation layer

SHAP ends with a ranked list of feature names. A user needs a sentence. The last stage of CARCI assembles the anomaly's facts - including that SHAP ranking - into a root-cause explanation and a recommended action, and it is worth being precise about which part of that is measured and which part is written.

9.1 What a root-cause explanation is made of

Part	Answers	Computed by
Service and date headline	What broke, and when	analysis.py:236
Observed cost vs baseline, increase %	How bad it is. The baseline is the mean of that service's OTHER days	_attach_service_context, analysis.py:102-138
Dominant region, resource, tag, usage type with cost share	Where the money went	_to_breakdown, analysis.py:33-49
Why it qualified: model flag, or the 75% / $5 rule, or reference-only	On what evidence	analysis.py:208-219
Top SHAP features	Which measurements drove the score	_get_feature_contributions, analysis.py:176-191
Recommended action	What to do next	The LLM - see 9.2

9.2 Two engines, one contract

The explanation text has two possible sources. The LLM is the primary one; the deterministic template is the offline fallback.

	LLM (primary)	Template (fallback)
Written by	Google Gemini, model gemini-3.5-flash-lite	Python f-strings
Code	carci/llm_reasoning.py	carci/analysis.py:207-242
Recommendation	Generated per anomaly from its own facts	One of five keyword branches on usage_type (analysis.py:194-204)
Runs when	A key is present and CARCI_LLM_REASONING is not 0	No key, no network, API error, or unparseable reply
Reported as	explanation_method = 'LLM (gemini-3.5-flash-lite)'	explanation_method = 'deterministic template'

> **The rule that keeps this honest**
> The model never computes a number. Every figure is calculated by the pipeline and passed in as JSON; the model decides only how to word those facts and what to recommend. This is the difference between an LLM feature that is safe in a financial report and one that is not.

9.3 The JSON contract

Up to 20 anomalies go out in one request (carci/llm_reasoning.py, MAX_RECORDS_PER_CALL). The payload for each is deliberately narrow - _to_payload passes only the facts the model is allowed to talk about, and deliberately does NOT pass the rule-based suggestion, so the returned action is the model's own reading rather than a paraphrase of a keyword match.

```
# Sent (one array element per anomaly):
{
  "index": 0,
  "service": "AmazonRDS",           "date": "05 January 2026",
  "observed_cost_usd": 1488.53,      "baseline_cost_usd": 687.92,
  "increase_pct": 116.38,            "impact_usd": 800.61,
  "driver_line_item_cost_usd": 241.33,
  "region": "us-west-2",             "region_share_pct": 22.65,
  "resource": "db-175a715d-...",     "usage_type": "NormalUsage",
  "top_features": ["cost", "autoscaling_count", "cost_per_usage"],
  "feature_attribution": [{"feature": "cost", "value": 241.33,
                           "shap_value": -1.958}, ...],
  "model_flagged": true,  "reference_only": false
}

# Returned, constrained by responseSchema so it parses rather than scrapes:
{ "explanations": [ { "index": 0,
                      "explanation": "...",
                      "recommended_action": "..." } ] }
```

The request sets responseMimeType='application/json' with an explicit responseSchema, and temperature 0.2. Structured output matters here for a practical reason: a free-text reply would have to be split back into per-anomaly chunks by string matching, which fails the moment the model formats something differently.

9.4 Real output, and how to check it

This is the actual explanation returned for the worked example in section 7. Compare each figure against the pipeline values in the payload above - that comparison is the exercise.

> **LLM output (gemini-3.5-flash-lite)**
> On 05 January 2026, AmazonRDS in the us-west-2 region experienced a cost anomaly flagged by the model. The observed cost reached 1488.53 USD compared to a baseline cost of 687.92 USD, resulting in an impact of 800.61 USD or a 116.38 percent increase. The specific driver line item cost was 241.33 USD under the NormalUsage usage type for resource db-175a715d-..., which is tagged as InternalTools and accounts for 22.65 percent of the region share. The top features driving the anomaly score are autoscaling count, cost, and cost per usage.
>
> Recommended action: Review the autoscaling count and cost per usage metrics for database resource db-175a715d-... in us-west-2 to determine if scaling limits can be tightened for InternalTools.

Every number there appears in the JSON that was sent. Contrast the recommendation with the template's for the same anomaly - 'Review the listed service, resource, tag, and usage type before increasing the budget' - which would fit any anomaly in any file. That gap is what the LLM layer buys.

9.5 Failure behaviour, and the limits to state plainly

- No key, no network, HTTP error, safety block or unparseable JSON: the record keeps its template text and template recommendation, and explanation_method says so. The analysis never fails, because no number in the report depends on the model.
- Both versions are always kept - template_explanation and template_recommendation - so the two can be shown side by side in class.
- Transient 429 / 500 / 503 responses are retried once, then give up.
- The layer adds roughly 5-10 seconds per analysis and sends service, region, resource ids and tag names to Google. Resource ids and tags are the sensitive part; raw billing rows are never sent. CARCI_LLM_REASONING=0 disables it entirely.
- Nothing in the code verifies the returned prose against the numbers that were sent. The prompt forbids invention and the observed output has been faithful, but a stricter implementation would re-check every figure that appears

in the text. Good exercise.

> **Attribution is not causation - say this explicitly to students**
> The 'main region / resource / usage type' lines are the largest contributors by COST SHARE, and the SHAP clause explains why the MODEL scored the row as it did. Neither proves what caused AWS to charge more. CARCI performs automated root-cause attribution and explanation; it does not perform causal inference, and there is no counterfactual anywhere in the codebase.

10. When SHAP does not run: the two fallbacks

CARCI never lets a missing explanation stop the analysis. There are three possible states, and the attribution_method string always tells you which one you are looking at.

attribution_method	When	Set at
SHAP TreeExplainer	Normal operation: shap installed and the call succeeded	isolation_forest_model.py:140
standardized feature impact fallback	shap missing, or the call raised, or the shape check failed	isolation_forest_model.py:147
not calculated	Fewer than 2 rows, or every row has the same cost, so no model is fitted at all	analysis.py:88-90

10.1 The standardized fallback, and its sign trap

The fallback is not SHAP. It is a standardized distance from the training median, scaled by the row's anomaly score: (value - median) / std, multiplied by anomaly_score. It answers a similar question ('which feature is most unusual for this row?') with much cruder machinery, and it costs nothing to compute.

Forcing the fallback on the same row used in section 7 shows a trap worth pointing out in class - the two paths use OPPOSITE sign conventions:

feature	shap_value (real SHAP)	shap_value (fallback)
data_transfer_gb	-2.6905	+1.0218
cost	-1.5110	+0.4577
usage_quantity	-1.0406	+0.3699
transfer_per_cost	-0.5500	+0.2351
autoscaling_count	-0.2044	+0.2121
daily_z	+0.1573	+0.0815
cost_per_usage	+0.0171	-0.0024

- Real SHAP: anomaly-driving features are NEGATIVE (they shorten the isolation path).
- Fallback: anomaly-driving features are POSITIVE (they are far above the median).
- The ranking by impact_strength is broadly unaffected - data_transfer_gb is first either way - which is why the UI keeps working. But the displayed SIGN flips meaning depending on which path ran, and nothing in the UI warns you.

> **Good exercise, and a genuine finding**
> Ask students to explain why the signs invert, then ask whether the sign column should be normalised so both paths agree. This is a real inconsistency in the codebase, not a contrived one - it is exactly the kind of detail a careful code review should catch.

10.2 Forcing each state for a demo

```
# 1. Normal SHAP path - just run the app and read the caption.
streamlit run app.py

# 2. Fallback path - hide the library from Python for one run:
python -c "import sys; sys.modules['shap']=None; import pandas as pd;\
  from carci.analysis import build_analysis_result;\
  df=pd.read_csv('dataset.csv').head(200);\
  print(build_analysis_result(df)['metadata']['attribution_method'])"

# 3. 'not calculated' - feed a file where every row has an identical cost.
```

11. Cost, limits and honest caveats

11.1 SHAP is the slowest thing in the project

The explanation costs far more than the detection it explains. Measured on this machine, fitting on 190 rows and explaining 200 rows:

Step	Time	Share
decision_function on 200 rows	75 ms	small
shap_values on 200 rows	2.17 s (10.9 ms/row)	~97%

Per-row cost grows with the training set, because max_samples=1.0 makes every tree deeper on a larger file. The project README records ~170 ms/row when fitted on the bundled 10,000-row dataset.csv, which works out to roughly 28 minutes of SHAP for a single upload - about 99% of total runtime, during which the Streamlit app appears to hang.

> **The obvious improvement, left as an exercise**
> CARCI explains every scored row, then displays at most a handful of anomaly cards. Explaining only the rows that will actually be shown would cut the dominant cost by orders of magnitude without changing a single displayed number. This is the highest-value change a student could make to the project.

11.2 Technical caveats to state plainly

Caveat	Detail
Path length, not money	SHAP values are in units of isolation path length. They rank causes; they do not decompose a dollar amount.
tree_path_dependent perturbation	The explainer runs with feature_perturbation='tree_path_dependent' (no background dataset is passed). Correlated features - and cost, cost_per_usage and usage_quantity are correlated by construction - share credit according to tree structure rather than an interventional baseline.
Explains the model, not reality	SHAP faithfully reports why THIS forest scored the row as it did. If the model is wrong, SHAP will confidently explain a wrong decision.
Local only	Each table explains one row. The project never averages |SHAP| across rows, so it has no global feature-importance view.
Ranking discards direction	top_ml_features sorts by absolute value, so a feature that argued AGAINST the anomaly can still be named as a 'strongest impact' in the sentence.
Fitted on the data it explains	The forest is fitted on the same upload it scores, so the scaler and the trees have both already seen the anomalies being explained.

12. Lesson plan, demos and exercises

12.1 A 75-minute session

Time	Segment	Material
0-5	Motivation: show a raw score, ask what to do with it	Section 1
5-15	Shapley values and why TreeExplainer	Section 2
15-25	Code tour, following the data forwards	Sections 3-5, Figure 1
25-35	Sign and additivity, with the verified numbers	Section 6, Figure 2
35-45	Worked example on screen in the running app	Section 7, Figure 3
45-58	Root cause: the JSON contract and the LLM layer	Section 9
58-68	Break it: force both fallbacks, discuss the sign trap	Section 10
68-75	Caveats and the performance exercise	Section 11

12.2 Live demo script

```
# 1. Start the app and upload a small CSV (200-500 rows keeps SHAP fast).
streamlit run app.py

# 2. On the summary page, read the caption: 'Attribution: SHAP TreeExplainer'.
# 3. Go to page 3. The caption above the anomaly list names the engine that
#    wrote the explanations: 'LLM (gemini-3.5-flash-lite)'.
# 4. Open any anomaly expander, open 'ML Feature Attribution'.
# 5. Point at the sign column and ask: which feature argued AGAINST the flag?
# 6. Read the explanation and check each number against the table above it.
# 7. Download the detailed CSV and find the feature_contribution_ rows.

# 8. Turn the LLM off and re-run to show the template fallback:
#    set CARCI_LLM_REASONING=0 in .env, then re-upload the same file.

# 9. Show that the CLI path skips SHAP and the LLM entirely, so it is fast:
python -m carci.isolation_forest_model dataset.csv
```

12.3 Exercises

#	Exercise	What it teaches
1	Compute base + sum(shap) for one row and reproduce score_samples with -2**(-path/c(n)).	Additivity is real, and the transform in between
2	Find a row where the highest-|SHAP| feature has a POSITIVE value. Is the generated sentence still honest?	Ranking by magnitude discards direction
3	Set explain=False in score_records and time a full analysis.	How much of the runtime is explanation
4	Force the fallback and compare the two attribution tables for the same row.	Sign conventions, and degradation vs failure
5	Remove daily_z from FEATURE_COLUMNS, re-run, and see how the remaining SHAP values redistribute.	Shapley values are relative to the feature set
6	Change _append_feature_attribution to explain only the displayed anomaly rows.	The real performance fix from section 11
7	Explain why service and region never appear in a SHAP table.	Model features vs business attribution

#	Exercise	What it teaches
8	Run the same file with CARCI_LLM_REASONING=1 and =0, and compare the two recommendations for the same anomaly.	What the LLM layer actually adds
9	Write a checker that extracts every number from the LLM explanation and asserts it appears in the JSON payload that was sent.	Verifying an LLM feature instead of trusting it
10	Read _to_payload and list what is deliberately NOT sent to the model, and why.	Prompt-surface minimisation

12.4 Exam-style questions with answers

Question	Answer
What does a negative SHAP value mean in CARCI?	The feature pushed the row toward being flagged, by shortening its isolation path.
Does SHAP train a second model?	No. TreeExplainer reads the trees of the fitted Isolation Forest itself.
Why can SHAP not tell you how many dollars a feature added?	It apportions the model's path-length output, which has no dollar interpretation.
What happens if shap is not installed?	The import inside the function fails, the except branch computes a standardized fallback, and attribution_method reports it.
Which features can appear in a SHAP table?	Only the seven in FEATURE_COLUMNS. Service, region, resource and usage_type are attributed separately by cost share.
Why is the Streamlit app slow but the CLI fast?	The CLI never calls _append_feature_attribution; SHAP is roughly 99% of the app's runtime.
Who computes the numbers in the root-cause paragraph?	The pipeline. The LLM receives them as JSON and only decides how to word them and what to recommend.
What does CARCI do if the Gemini API is unreachable?	Keeps the deterministic template paragraph and the rule-based recommendation, and reports 'deterministic template' as the explanation_method.
Is this causal analysis?	No. It is attribution: cost share for the business dimensions, SHAP for the model features. Nothing in the codebase tests a counterfactual.

13. Glossary

Term	Meaning in this project
XAI	Explainable AI: techniques that make a model's output interpretable to a human.
SHAP	SHapley Additive exPlanations. Assigns each feature a signed contribution to one prediction.
Shapley value	The game-theoretic fair share of a payout, averaged over all orderings of the players.
TreeExplainer	The SHAP algorithm specialised for tree ensembles; exact and fast.
Base value	The average model output over the training data. Here 10.848936 in path-length units.
Path length	How many splits it takes a tree to isolate a row. Short = anomalous.
Local accuracy	base value + sum of SHAP values = the model output for that row.
impact_strength	abs(shap_value); the sort key used to rank features in CARCI.
attribution_method	Provenance string: which of the three explanation states produced the numbers.
Isolation Forest	The unsupervised detector: 1000 trees, contamination 0.05, random_state 123.
decision_function	sklearn's anomaly score. Negative = outlier. CARCI stores the negation as anomaly_score.
Fallback attribution	The non-SHAP approximation used when the SHAP call cannot complete.
explanation_method	Which engine wrote the paragraph: the LLM, or the deterministic template.
Structured output	Constraining an LLM reply to a JSON schema so it can be parsed instead of scraped. Used for every explanation request.
Root-cause attribution	Localising an anomaly to a service, day, region, resource and usage type, and naming the measurements that drove the score. Not the same as proving causation.

Source files referenced in this guide

File	Sections
carci/isolation_forest_model.py	1, 3, 5, 6, 10, 11
carci/shared.py	3, 4, 6
carci/analysis.py	1, 3, 8, 9, 10
carci/reporting.py	1, 3, 8
carci/llm_reasoning.py	3, 8, 9
app.py	1, 3, 8, 9, 12
requirements.txt	3
README.md	11

All measurements in this guide were produced by running the CARCI pipeline on the first 200 rows of 'data set/test_2_obvious_anomalies.csv' with shap 0.50.0 and Python 3.13.5. Line numbers refer to the project source as of this document's creation; re-check them after editing the files.